

Chapter 15

Using Stories with Scrum

User stories originated as part of Extreme Programming. Naturally, stories fit perfectly with the other practices of Extreme Programming. However, stories also work well as the requirements approach for other processes.

In this chapter we'll look at Scrum, another agile process, and will see how stories can be integrated as an important part of Scrum.¹ Terms that are part of the Scrum lexicon will be italicized when first used.

Scrum Is Iterative and Incremental

Like XP, Scrum is both an iterative and an incremental process. Since these words are used so frequently without definition, we'll define them.

An iterative process is one that makes progress through successive refinement. A development team takes a first cut at a system, knowing it is incomplete or weak in some (perhaps many) areas. They then iteratively refine those areas until the product is satisfactory. With each iteration the software is improved through the addition of greater detail. For example, in a first iteration a search screen might be coded to support only the simplest type of search. The second iteration might add additional search criteria. Finally, a third iteration may add error handling.

A good analogy is sculpting. First the sculptor selects a stone of the appropriate size. Next the sculptor carves the general shape from the stone. At this point one can perhaps distinguish the head and torso, and discern that the finished work will be of a human body rather than a bird. Next, the sculptor refines her work by adding detail. However, the sculptor is unlikely to look on any one area as complete until the entire work is complete.

1. For complete coverage of Scrum, see *Agile Development with Scrum* (Schwaber and Beedle 2002).

An incremental process is one in which software is built and delivered in pieces. Each piece, or increment, represents a complete subset of functionality. The increment may be either small or large, perhaps ranging from just a system's login screen on the small end to a highly flexible set of data management screens. Each increment is fully coded and tested, and the common expectation is that the work of an iteration will not need to be revisited.

An incremental sculptor would pick one part of her work and focus entirely on it until it's finished. She may select small increments (first the nose, then the eyes, then the mouth, and so on) or large increments (head, torso, legs and then arms). However, regardless of the increment size, the incremental sculptor would attempt to finish the work of that increment as completely as possible.

Scrum and XP are both incremental and iterative. They are iterative in that they plan for the work of one iteration to be improved upon in subsequent iterations. They are incremental because completed work is delivered throughout the project.

The Basics of Scrum

Scrum projects progress through a series of thirty-day iterations called *sprints*. At the start of each sprint the team determines the amount of work it can accomplish during that sprint. Work is selected from a prioritized list called the *product backlog*. The work the team believes it can complete during the sprint is moved onto a list called the *sprint backlog*. A brief daily meeting, the *daily scrum*, is held to allow the team to inspect its progress and to adapt as necessary. Graphically, Scrum is as shown in Figure 15.1, which is adapted from Ken Schwaber's website at www.controlchaos.com.

The Scrum Team

A Scrum team typically consists of four to seven developers. While a Scrum team may involve specialist developers—testers and database administrators, for example—a Scrum team shares a “we’re all in this together” attitude. If there is testing to be done and there is no dedicated tester available, then someone else does the testing. All work is owned collectively. Scrum teams are self-organizing. That is, there is no management directive stating that Mary codes and Bill tests. Because of this, role names such as programmer, architect and

tester are generally not used on Scrum teams. Everything about how a team accomplishes its work is left up to the team.

This core team is supplemented by two key people: the *product owner* and the *ScrumMaster*. A Scrum product owner is essentially the customer of Extreme Programming. The product owner is largely responsible for placing items onto and prioritizing the product backlog list of needed functionality. The ScrumMaster is similar to a project manager except that the role is much more one of leadership than of managing. Because Scrum teams are self-organizing and given great leeway in how they complete the work of a sprint, the project's ScrumMaster serves the team rather than directs it. A ScrumMaster typically serves her team by removing obstacles to progress and by helping the team follow the few simple rules of Scrum.

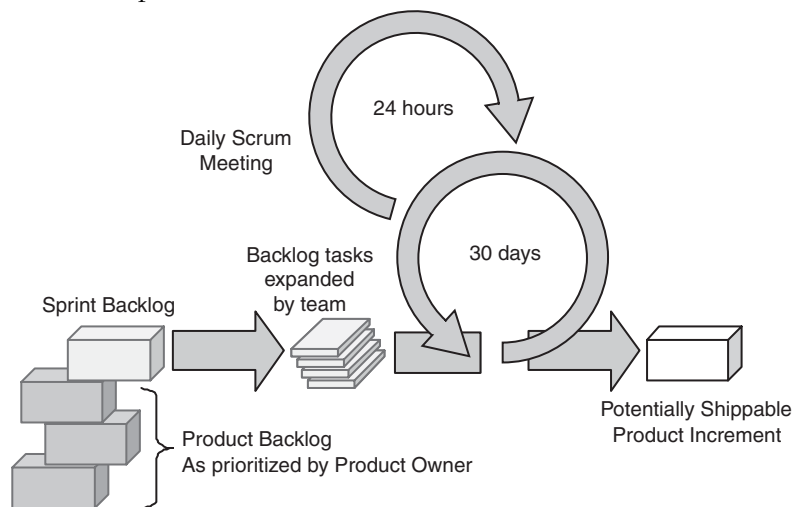


Figure 15.1 Graphical representation of the Scrum process.

The Product Backlog

The product backlog is the master list of all functionality desired in the product. When a project is initiated there is no comprehensive effort to write down all foreseeable features. Typically, the product owner and team write down everything obvious, which is almost always more than enough for a first sprint. The product backlog is then allowed to grow and change as more is learned about the product and its customers.

An example product backlog from a real project appears as shown in Table 15.1. As you can see from this table, backlog items can be technical tasks (“Refactor the Login class to throw an exception”) or more user-centric (“Allow undo on the setup screen”).

The product owner sorts the product backlog into priority order. Even better, the product owner is allowed (actually encouraged) to shuffle the product backlog items as priorities change from month to month.

Table 15.1 *A sample product backlog list.*

Number	Description
1	Finish database versioning
2	Get rid of unneeded Java in the database
3	Refactor the Login class to throw an exception
4	Add support for concurrent user licensing
5	Add support for evaluation licenses
6	Support wildcards when searching
7	Save user settings
8	Allow undo on the setup screen

The Sprint Planning Meeting

A *sprint planning meeting* is held at the start of each sprint. The meeting usually lasts up to a full day and is attended by the product owner, the ScrumMaster and the entire team of developers. Also attending may be any interested and appropriate management or customer representatives.

During the first half of the sprint planning meeting, the product owner describes the highest priority remaining features to the team. The team asks enough questions that during the second half of the meeting they can determine which items they will move from the product backlog to the sprint backlog.

The product owner does not have to describe every item being tracked on the product backlog. Depending on the size of the backlog and the velocity of the team it may be sufficient to describe only the high priority items, saving the discussion of lower priority items for the next sprint planning meeting. Typically, the Scrum team will provide guidance when they start to get further into the backlog list than they know can be done in the upcoming sprint.

Collectively the team and the product owner define a *sprint goal*, which is a short description of what the team plans to achieve during that sprint. In a

sprint review meeting held at the end of the sprint, the team will assess their success against the sprint goal, rather than against each specific item selected from the product backlog.

During the second half of the sprint planning meeting, the team meets separately to discuss what they heard and decide how much they can commit to during the coming sprint. Conceptually, the team starts at the top of the prioritized product backlog list and draws a line after the lowest of the high priority tasks they feel they can complete. In practice it is not unusual to see a team select, for example, the top five items and then two items from lower on the list that are associated with the initial five. In some cases there will be negotiation with the product owner, but it will always be up to the team to determine how much they can commit to completing.

The Main Rules of Scrum

A sprint planning meeting is held at the start of each sprint.

Each sprint must deliver working and fully tested code that demonstrates something of value to end-users or the customer.

The product owner prioritizes the product backlog.

The team collectively selects the amount of work brought into the sprint.

Once a sprint begins, only the team may add to the sprint backlog.

The product backlog may be added to or reprioritized at any time.

A short scrum meeting is held every day. Each project participant answers: What did you do yesterday? What will you do today? What obstacles are in your way?

Only active participants in the sprint (not interested observers or removed stakeholders) may speak during the daily scrum meeting.

The result of a sprint is demonstrated at a sprint review meeting at the end of the sprint.

Working software is demonstrated during the sprint review. No slideshows are allowed.

No more than two hours may be spent preparing for the sprint review.

Once the sprint starts, only the team may add work to the sprint. The CEO cannot approach the team and ask for something the product owner left out. A salesperson cannot ask for one more feature for a special customer. And the product owner cannot change her mind and ask for additional features. The only time work may be added to a sprint is if the team finds itself ahead of schedule. At that time the team can ask the product owner to help identify an additional item or two.

In exchange for their commitment to complete the work selected for the sprint, the team receives a commitment from the organization that it will not change the contents of sprint for the duration of the sprint. If something so significant happens that the organization feels it needs to change the sprint, then the current sprint is aborted and a new sprint is started by beginning with another sprint planning meeting.

As the team selects items from the product backlog, they expand them into the more task-based sprint backlog. Each item on the product backlog may expand into one or more items on the sprint backlog, allowing the team to more effectively share the work.

The Sprint Review Meeting

Each sprint is required to deliver a *potentially shippable product increment*. This means that at the end of each month-long sprint the team has produced a coded, tested and usable piece of software. The software must be potentially shippable, meaning that the organization could choose to ship the software to customers (or use it internally) if enough new functionality is included to justify the overhead of shipping or deploying the new version. A commercial software distributor, for example, would probably not choose to ship a new version every month because of the upgrade hassles that could cause their customers. However, with Scrum the developers are required to produce a potentially shippable version each month.

At the end of each sprint, a sprint review meeting is held. During this meeting the team shows what they accomplished during the sprint. Typically this takes the form of a demo of the new features.

The sprint review meeting is intentionally kept very informal, typically with rules forbidding the use of PowerPoint slides and allowing no more than two hours of preparation time for the meeting. A sprint review meeting should not become a distraction or significant detour for the team; rather, it should be a natural result of the sprint.

The entire team as well as the product owner and ScrumMaster participate in the sprint review meeting. Others who are interested in the project (such as management, customers or engineers from other projects) may attend the sprint review if they desire.

During the sprint review meeting the project is assessed against the sprint goal that was previously determined during the sprint planning meeting. Ideally,

the team has completed each item planned for the sprint but it is more important that they achieve the overall goal of the sprint.

The Daily Scrum Meeting

Scrum is probably the first documented process to include a short daily meeting (Beedle 1999), which in Scrum is called “the daily scrum.” The idea, however, has quickly spread to many of the other agile processes such as XP and Feature-Driven Development. Whenever possible we want to choose the least time-consuming and the least-intrusive method of gathering and sharing project information. For many purposes, the daily scrum achieves exactly that goal.

The daily scrum is typically held as early as possible each day but after the entire team has arrived for work. This is typically 9:00 or 9:30. Everyone on the team is required to attend, including programmers, testers, the product owner and the ScrumMaster. The meeting is kept short: usually 15 minutes or less and never longer than 30. To keep the meeting short, some teams require participants to stand.

During the daily scrum each team member answers the following three questions:

1. What did you do yesterday?
2. What will you do today?
3. What obstacles are in in your way?

It is important that the daily scrum does not come across as a grilling by the ScrumMaster. One of the goals of the meeting is to have each developer make commitments in front of his or her peers. Commitments are not made to the manager or the company but to each other.

The daily scrum should stick as closely as possible to the three questions above, and should not move off into designing parts of the system or resolving issues that were raised. Such items are noted during the meeting but are resolved afterwards. It's OK to identify a subset of the team that will meet to resolve an issue, but do not resolve issues during the daily scrum. For example, suppose someone asks whether we should start using the recently released version 5.0 of our vendor's application server. In that case, we agree that right after the daily scrum another meeting will occur between:

- the technical architect, who can assess the technical impact on using the new application server

- the product owner, who comes from the Marketing department and will be in the best position to decide if our customers will deploy the old or new application server
- a representative from the test team who can assess the impact on her group

The ScrumMaster is on hand as a facilitator and to make sure the meeting stays focused on the three questions and moves along at a brisk pace. The product owner is present because she will ideally have work to report on just like anyone else (“I finished writing tests for the ‘Add book to shopping cart’ story and today I’m going to do some market research on which cards we should accept. I should be able to finish that by the end of the day.”).

One benefit to holding daily meetings is that they can serve as random checkpoints for senior managers or anyone else interested in the state of the project. By consistently meeting at the same time of day and extending a general “join us whenever you’re interested” invitation, the team may be able to get out of more onerous meetings such as monthly project reviews. However, if non-team members are invited to the daily scrums, be sure to establish a rule that only people directly on the project can ask questions during the meeting. So, the Big Boss can attend and listen to her heart’s content. She cannot, however, redirect the meeting by asking questions.

The daily scrum provides everyone on the team a quick daily snapshot of where things stand on the project. This makes it the perfect time for the team to reconsider current assignments. For example, suppose Randy reports that the story he’s working on is taking much longer than he expected and Andrew reports that he is significantly ahead of schedule. In this case it may be appropriate for Andrew to spend the day paired with Randy and working on Randy’s tasks or for Andrew to outright take on responsibility for some of Randy’s tasks.

The ScrumMaster must walk a fine line during the daily scrums. She needs to keep the pace brisk but cannot let the meetings feel as though they are for her benefit alone. One question that should never be asked is “How much time do you have left on the ‘Order a book’ story?” This information is extremely important but if it is asked for during the daily scrum then the daily scrum will become all about estimates and numbers. Separate from the daily scrum meeting, I have the team update their estimates on a communal whiteboard or in the software we’re using if we’re not collocated.

Adding Stories to Scrum

Having described the general Scrum approach we now turn our attention to how Scrum can be improved by combining it with the use of stories.

Stories and the Product Backlog

I have had great success expressing Scrum backlog items in the form of user stories (Cohn 2003). Rather than allowing product backlog items to describe new features, issues to investigate, defects to be fixed, and so on, the product backlog is constrained to only user stories. Each story in the product backlog must describe some item of value to a user or to the product owner.

By constraining the product backlog to only user stories it becomes much easier for the product owner to prioritize the backlog. With all backlog items in terms she can understand, it becomes easy for the product owner to make decisions to trade one feature for another.

As with XP, in Scrum it is not important for the product owner to identify all of the requirements up front. However, there is often a benefit to jotting down as many of them as possible at the outset. Scrum has no prescribed, or even recommended, approach to initially stocking the product backlog. Traditionally this has happened through an unstructured discussion between the product owner, ScrumMaster, and one or more developers. However, I've found that by first identifying user roles and then focusing on the stories for each user role, Scrum is combined with a powerful requirements identification technique.

Stories in the Sprint Planning Meeting

During the sprint planning meeting, the product owner and team discuss the top priority items on the product backlog. The team then identifies the items they will commit to completing during the sprint. They then break those tasks into smaller tasks as necessary so that developers can sign up for tasks as the sprint progresses.

Stories force each product backlog item to each deliver value that the customer can assess. Because of this I have found that sprint planning meetings are easier to conduct and go faster than when the team is forced to explain the benefit of technology-driven backlog items such as “refactor the Login class to throw an exception.”

Stories also fit well into sprint planning because, as we saw in Chapter 10, “Planning an Iteration,” stories may be readily disaggregated into their constituent tasks.

Stories in the Sprint Review Meeting

The use of stories benefits Scrum during the sprint review meeting because stories make it simpler to assess what parts of a sprint have been completed. On a Scrum project that uses a random collection of technical tasks, requirements, issues and bug fixes as its product backlog, it can be difficult for the team to demonstrate that each item made its way into the product built during that sprint. When the entire product backlog is composed of stories describing items of customer or user value, it is generally easier to demonstrate those items.

Stories and the Daily Scrum Meeting

I have found stories beneficial to daily scrum meetings because they help ensure focus remains on customer and end-user desires. Because there is no upfront requirements or analysis phase that precedes a sprint, sprints are started with only a partial understanding of exactly what will be built. The team may know they are planning to add a search screen but they may not know what fields will be searchable, how search criteria may be combined, and so on. Stories are useful because they remind the team about the intent behind what is being developed. In the midst of a sprint the team can use the story (as well as ongoing discussions with the product owner about the story) to determine whether they have gone far enough, or perhaps too far, in programming a particular story.

A Case Study

As an example of adding stories to Scrum, let’s consider a project I was involved in. The company, let’s call them Cosmodemonic Biotech for convenience, was a small but publicly-traded developer of software for the life sciences industry. Cosmodemonic Biotech had just completed a nine-month development effort to introduce a new product for human geneticists. Shortly after the new product was delivered to its initial beta site, the company announced that it was being acquired.

The acquiring company was interested in the list of customers that came along with the product. However, they decided that the software would need to be rewritten for a variety of reasons, including:

- the client technologies of the original product (HTML) did not fit well with the strategic vision of the new owner
- the target market for the product changed from very large pharmaceutical companies with multi-terabyte databases to academic research labs and small biotech companies with much smaller databases
- a relatively poor implementation throughout much of the original code

The original product had been developed in a very strict waterfall manner over a period of nine months with a team that was as large as 100 developers. The new product would need to deliver essentially the same functionality but would have a team size of no more than seven.

To achieve this, we used Scrum augmented with user stories as described in this chapter. By all measures the project was a success. It took the Scrum team twelve months to complete what the waterfall team had done in nine. However, because the Scrum team never exceeded seven people, including the product owner and ScrumMaster, the project took 54 person-months to complete. The waterfall version had taken 540 person-months to complete.

The waterfall team produced 58,000 non-comment Java source statements. The Scrum with stories team did more with less, and produced 51,000. This means the waterfall team produced 120 Java source statements per person-month while the Scrum team produced 840 per person-month. These comparisons are shown in Table 15.2.

Table 15.2 *Comparison of the two approaches to the same project.*

	Waterfall	Scrum with Stories
Use Case Pages	3,000	0
Stories	0	1,400
Calendar Months	9	12
Person Months	540	54
Lines of Java Code	58,000	51,000
Lines of Java Code / Person-Month	120	840

Summary

- Scrum is an iterative and an incremental process.
- Scrum projects progress in a series of thirty-day iterations called sprints.

- A ScrumMaster fills the project manager role but is more leader and facilitator than manager.
- A typical Scrum team includes four to seven developers.
- The product backlog is a list of all desired features that have not yet been either added to the product or planned for the current sprint.
- The sprint backlog is a list of tasks the team has committed to for the current sprint.
- XP's customer role is played by the Scrum product owner.
- The product owner prioritizes the product backlog.
- At the start of the sprint, the team selects what and how much work they commit to completing during the sprint.
- Brief daily scrum meetings are held. During these meetings each team member says what she completed yesterday, what she'll complete today, and identifies any obstacles in her way.
- Each sprint is responsible for producing a potentially shippable product increment.
- At the end of sprint, the team demonstrates the software it created in a sprint review meeting.

Questions

- 15.1 Describe the differences between an incremental and an iterative process.
- 15.2 What is the relationship between the product backlog and the sprint backlog?
- 15.3 What is meant by a potentially shippable product increment?
- 15.4 Who is responsible for prioritizing work and for selecting the work the team will perform during a sprint?
- 15.5 What questions are answered by each team member at the daily scrum?